МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ

Циклова комісія інформаційних технологій.

КУРСОВИЙ ПРОЕКТ
(РОБОТА)

Програмування
(назва дисципліни)

на тему: Розробка програмного додатку для сворення опитувань та тестів

Студента (ки) IV курсу 46-ІС групи спеціальності 151 Автоматизація та комп'ютерно-інтегровані технології

спеціалізації 5.151.1 Обслуговування інтелектуальних інтегрованих систем

Керівник: викладач ***Васильєв М.В.***

Національна шкала _____
Кількість балів: _____Оцінка: ECTS _____

Члени комісії _____ (**Васильєв** М.В.)
(підпис)
_____ (_____)
(підпис)
_____ (_____)
(підпис)

м. Харків – 2024 рік

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ

РОЗГЛЯНУТО
на засіданні циклової комісії Голова
циклової комісії
Марина ВЕЛИЧКО
«24» жовтня 2024 р.

ЗАТВЕРДЖЕНО
Зав. відділення
Олеся ТВЕРДОХЛІБОВА
«25» жовтня 2024 р.

Завдання

на курсовий проєкт студенту гр. 46-ІС
спеціальність/освітньо-професійна програма 151 Автоматизація та комп'ютерно-інтегровані технології/5.151.1 Обслуговування інтелектуальних інтегрованих систем

Гробова Юрія Руслановича

Тема: Розробка програмного додатку для створення опитувань та тестів.
Термін виконання роботи з 12.11.2024 р. по 31.03.2025 р.
Вхідні дані для проектування: (Варіант №14)

Розробка програмного додатку для створення опитувань та тестів. Програма повинна мати можливість проводити опитування та тестування користувачів за поданими тестами, а також мати можливість збереження відповідей користувачів. Інформація про тести зберігається в базі даних.

Література та посібники для проектування:

1. Head-First Python, 2nd edition: Paul Barry. - Sebastopol, California, U.S.: O'Reilly Media, 2016. – 622 с.
2. Think Python: How to Think Like a Computer Scientist, 2nd edition: Allen B. Downey. - Sebastopol, California, U.S.: O'Reilly Media, 2015. – 292 с.
3. Clean Code: A Handbook of Agile Software Craftsmanship: Robert C. Martin. - London, England: Pearson, 2008. – 464 с.
4. Python.org: веб-сайт. URL: https://www.python.org (дата звернення: 01.09.2024)
5. Python Tutorial: веб-сайт. URL: https://www.w3schools.com/python/ (дата звернення: 01.09.2024)
6. Learn to become a modern Python developer: веб-сайт. URL: https://roadmap.sh/python/ (дата звернення: 01.09.

Зміст розрахунково-пояснювальної записки

№ п/п	Зміст	Планований термін виконання	Фактичний термін виконання	%
43.	Вступ	30.11.2024	30.11.2024	5
44.	Аналіз задачі, засобів та методів її вирішення	13.12.2024	13.12.2024	15
45.	Проектування загального алгоритму роботи програми	18.12.2024	18.12.2024	40
46.	Розробка програмного забезпечення	02.03.2025	02.03.2025	80
47.	Керівництво користувача	09.03.2025	09.03.2025	90
48.	Висновки	15.03.2025	15.03.2025	95
49.	Список використаних джерел	29.03.2025	29.03.2025	100

Керівник курсової роботи _____Микола ВАСИЛЬЄВ

Завдання до курсової роботи

Одержав(ла) студент(ка) гр. 46-ІС _____(_____)

Дата «12» листопада 2024 р.

Реферат

Об’єктом розробки є програма для створення, проведення та аналізу опитувань і тестів.

Мета розробки – розробка програмного забезпечення для зручного створення та управління тестами й опитуваннями.

У процесі роботи проведено розробку блок-схеми алгоритму роботи програми та окремих функцій, створено програму, яка реалізує цей алгоритм.

Основні конструктивні та техніко-економічні показники: висока надійність, зручність використання, оптимізованість.

Розроблений застосунок може використовуватися для створення опитувань та тестів у навчальних, дослідницьких або корпоративних цілях.

Python, PyCharm, IDE, опитування, тестування, аналіз результатів, функція, метод, клас, оператор, змінна, параметр.

Перелік скорочень

МП – Мова програмування;

ПЗ – Програмне забезпечення;

IDE – Інтегроване середовище розробки;

Зміст

Реферат 4
Перелік скорочень 5
ВСТУП 7
ОСНОВНА ЧАСТИНА 8
1.Аналіз задачі, засобів та методів її вирішення 8
1.1 Python 8
1.2 Історія створення МП Python 9
1.3 Серкдовище розробки Pycharm 9
2. Проектування загального алгоритму роботи програми 11
3. Розробка програмного забезпечення 14
3.1 Опис структури бази даних для тестувальної системи 14
3.2 Реалізація програмного коду 18
3.3 Адміністративна панель 21
3.4 Робота з базою даних 24
4. Керівництво користувача 29
4.1 Опис роботи з програмою для користувача 29
4.2 Опис роботи з програм для адміністратора 31
ВИСНОВКИ 34
СПИСОК ЛІТЕРАТУРИ 35
ДОДАТКИ 36
Додаток А 36
Додаток Б 40
Додаток В 47

ВСТУП

Розвиток інформаційних технологій потребує ефективних інструментів для організації тестування та збору аналітичних даних. Саме тому створення програмного додатку для тестування та опитувань є актуальним завданням. Цей додаток дозволяє проводити оцінку знань, отримувати результати в реальному часі та зберігати їх для подальшого аналізу.

У сучасному освітньому та корпоративному середовищі автоматизовані системи тестування значно покращують ефективність навчального процесу та професійної атестації. Завдяки інтеграції з базами даних та використанню графічного інтерфейсу, такі додатки забезпечують простоту у використанні, швидкість перевірки результатів і можливість статистичного аналізу відповідей.

У даній роботі буде реалізовано систему, яка використовує Python як основну мову програмування, PyCharm як середовище розробки та CSV для роботи з даними. Програмний продукт повинен мати гнучкий інтерфейс для створення тестів та опитувань, можливість їх редагування та управління відповідями користувачів.

ОСНОВНА ЧАСТИНА

1. Аналіз задачі, засобів та методів її вирішення

Розробка даного проєкту буде здійснюватися за допомогою мови програмування Python у середовищі PyCharm, яке надає всі необхідні інструменти для створення ефективного та оптимізованого застосунку.

Для зберігання даних про рекламні кампанії, бюджети та результати використовуватиметься CSV-файл. Він слугуватиме аналогом бази даних, з якою програма буде працювати, виконувати необхідні обчислення та формувати звіти на запит користувача.

1.1 Python

Python — це інтерпретована, інтерактивна, об'єктно-орієнтована та високорівнева мова програмування загального призначення. Вона підтримує динамічну сувору типізацію та автоматичне керування пам’яттю, що сприяє підвищенню продуктивності розробника, покращенню читабельності коду та забезпеченню легкого портування програм.

Мова має потужні високорівневі структури даних і пропонує простий, але ефективний підхід до об’єктно-орієнтованого програмування. Завдяки зрозумілому синтаксису та динамічній типізації Python є ідеальним вибором для написання сценаріїв і швидкої розробки додатків у різних сферах і на багатьох платформах.

Python активно використовується для створення вебзастосунків, програмного забезпечення та рішень у сфері машинного навчання. Він широко застосовується у таких компаніях, як Google, Instagram, Facebook, IBM, NASA, Dropbox, Netflix тощо.

Інтерпретатор Python можна легко розширювати новими функціями та типами даних, реалізованими на C або C++ (або інших мовах, що можуть взаємодіяти з C). Також ця мова підходить для розширення можливостей кастомних додатків.

1.2 Історія створення МП Python

Python був створений у 1991 році голландським програмістом Гвідо ван Россумом, співробітником інституту CWI. У процесі розробки він запозичив деякі ідеї з мови ABC, яка була орієнтована на навчання програмуванню, і в розробці якої він також брав участь. Через це ABC часто називають попередником Python.

Цікаво, що назва "Python" не має відношення до змій. Гвідо ван Россум був великим шанувальником британського комедійного шоу 70-х років "Літаючий цирк Монті Пайтона", тому вирішив назвати свою мову саме так.

Завдяки простоті, читабельності коду та широким можливостям Python швидко набув популярності та зараз є однією з найпопулярніших мов програмування у світі. Його розвиток активно підтримується великою міжнародною спільнотою розробників. Нова функціональність і вдосконалення мови впроваджуються через систему PEP (*Python Enhancement Proposal*) — це спеціальні документи, в яких розробники обговорюють і затверджують зміни перед їх офіційним впровадженням.

Ключові фактори успіху Python — це його універсальність, адаптивність до різних сфер (від веброзробки до штучного інтелекту), а також активне оновлення та підтримка спільноти.

1.3 Середовище розробки PyCharm

Для розробки програмного забезпечення було обрано інтегроване середовище розробки (IDE) PyCharm. Це потужний інструмент, створений компанією JetBrains, який широко використовується для написання програм на Python. PyCharm підтримує операційні системи Windows, macOS та Linux, надаючи розробникам зручний та ефективний робочий простір.

PyCharm володіє багатьма функціями, які спрощують процес написання, налагодження та тестування коду, зокрема:

- Редактор коду з підсвічуванням синтаксису, автодоповненням, лінтингом і рефакторингом
- Налагоджувач із можливістю встановлення точок зупину, покрокового виконання коду та перегляду значень змінних
- Вбудований термінал, який дозволяє запускати команди безпосередньо в середовищі розробки
- Інтеграція з системами контролю версій (Git, GitHub, Mercurial, SVN), що спрощує командну роботу та керування кодом
- Механізм тестування, який підтримує написання та запуск юніт-тестів для перевірки працездатності програм

Створення та запуск проєкту в PyCharm

Щоб почати роботу в PyCharm, необхідно спочатку створити новий проєкт. Для цього слід виконати наступні кроки:

1) Відкрити меню "Файл" → "Створити" → "Проєкт"

У вікні, що відкриється, вибрати тип проєкту, вказати його назву та місце розташування

2) Після створення проєкту можна одразу переходити до написання коду

3) Редактор PyCharm автоматично підсвічує синтаксис, допомагає виправляти помилки в режимі реального часу та пропонує автодоповнення коду, що значно прискорює процес розробки.

Запустити програму можна кількома способами:

- Натисканням кнопки "Виконати" на панелі інструментів
- Використанням комбінації клавіш Shift + F10

Для налагодження коду в PyCharm є спеціальний debugger (налагоджувач), який дозволяє переглядати виконання програми покроково, аналізувати значення змінних та знаходити помилки.

2. ПРОЕКТУВАННЯ ЗАГАЛЬНОГО АЛГОРИТМУ РОБОТИ ПРОГРПМИ

Розробка програми складається з двох основних етапів:

1.Розробка логіки роботи з даними

Створено окремий файл database.py, який містить клас Database.

Цей клас забезпечує взаємодію з базою даних TestsDB та включає методи для отримання тестів, питань, відповідей, а також збереження та пошуку результатів.

Програма використовує SQL-запити для роботи з інформацією про користувачів та їхні відповіді.

2. Розробка інтерфейсу програми

Файл main.py – клієнтська частина для проходження тестів.

Файл main2.py – адміністративна панель для перегляду результатів.

Обидві частини побудовані на бібліотеці Tkinter, що дозволяє створити зручний графічний інтерфейс.

На першому етапі було продумано структуру програми та розробленометоди для роботи з базою даних, включаючи зчитування, редагування, збереження та аналіз відповідей.

На другому етапі реалізовано запуск програми, створення тестового CSV-файлу, у якому містяться стартові дані. Також передбачена взаємодія з користувачем через графічний інтерфейс, що забезпечує вибір тесту, введення відповідей та обробку результатів.

Алгоритм роботи користувача

Ця блок-схема описує основні кроки проходження тесту звичайним користувачем:

Відображення списку доступних тестів.

Користувач обирає тест.

Завантажуються питання та можливі варіанти відповідей.

Введення імені користувача. Якщо ім'я не введено, програма повертає

користувача до цього кроку.

Відображення тестових питань.

Вибір користувачем відповідей.

Збереження відповідей у базі даних.

Відображення повідомлення про успішне завершення тесту.

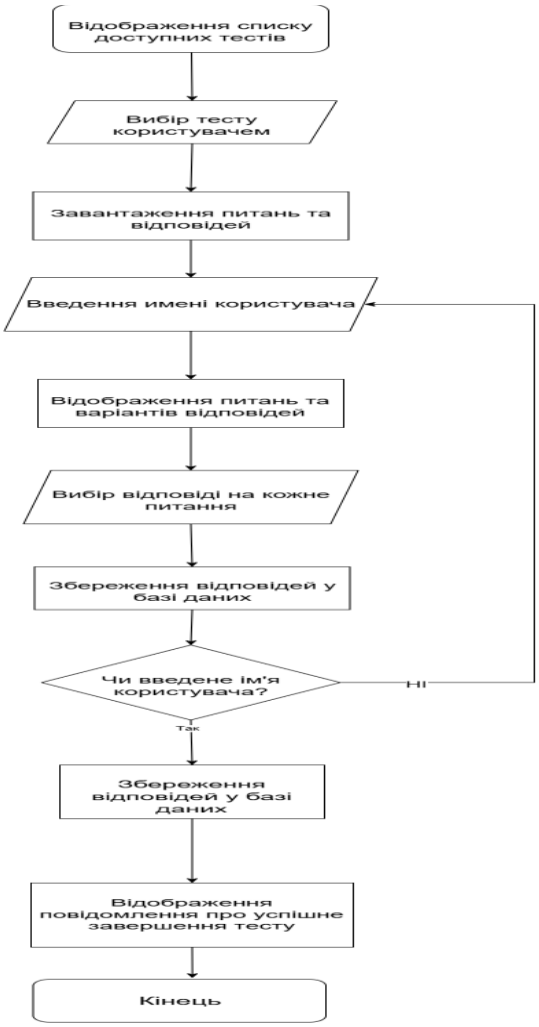

Рис. 2.1– Алгоритм роботи застосунку користувача

Алгоритм роботи адміністративної панелі

Дана блок-схема відображає логіку роботи адміністратора з системою тестування. Процес взаємодії включає наступні кроки:

Відображення панелі адміністрування.

Адміністратор може вибрати різні дії:

Пошук користувача. Якщо користувача знайдено – виводяться його результати тестування.

Перегляд невдалих спроб тестування. У цьому випадку система відображає список користувачів, які не змогли правильно відповісти.

Перегляд кількості спроб кожного користувача.

Вихід із програми. Якщо адміністратор вибирає цю опцію, робота системи завершується. В іншому випадку виконується повернення до панелі адміністрування.

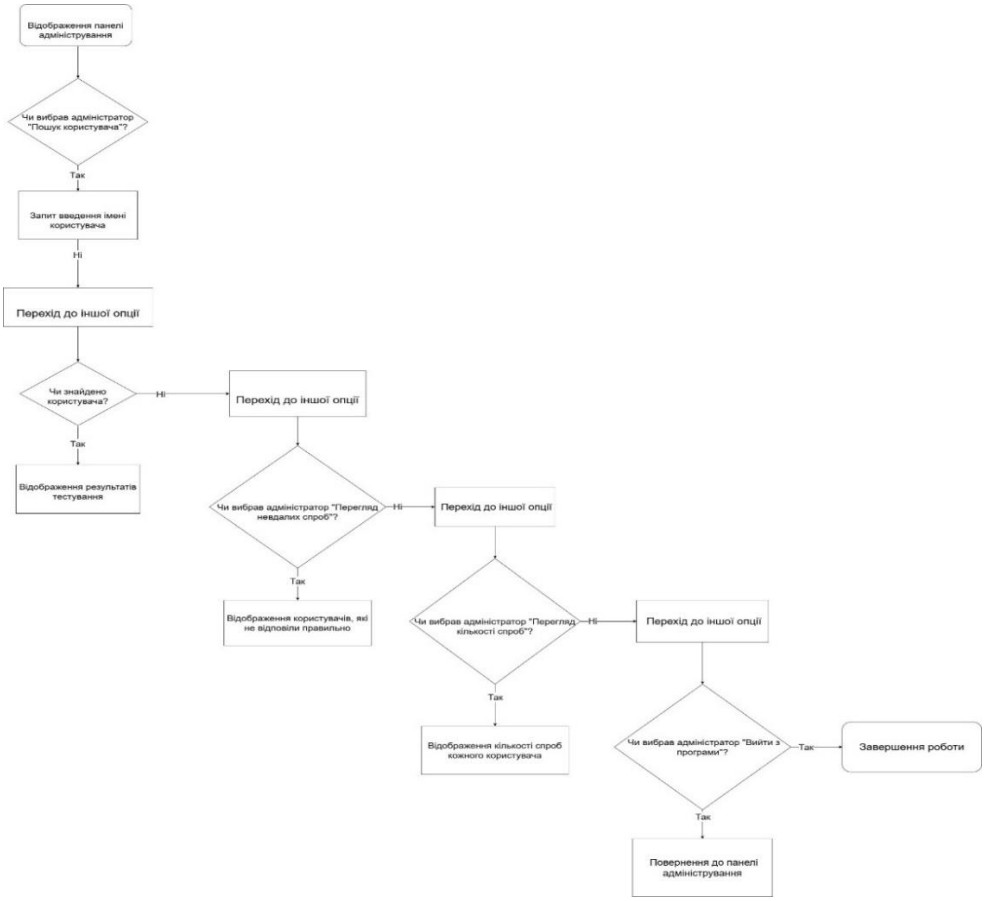

Рис. 2.2 – Алгоритм роботи адмін панелі

3. РОЗРОБКА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

3.1Опис структури бази даних для тестувальної системи

У процесі розробки моєї тестувальної системи я забезпечив ефективне зберігання та обробку інформації про користувачів, тести, запитання та їхні відповіді. Для цього я використовую базу даних Microsoft SQL Server 2014, яка складається з кількох взаємопов'язаних таблиць. У цьому розділі я розглядаю структуру бази даних, призначення її таблиць і спосіб взаємодії між ними.

Основна функція цієї бази даних – це зберігання тестів і відповідей користувачів, обробка отриманих даних та надання можливості аналізу результатів тестування. Завдяки такій організації я можу ефективно керувати тестами, відстежувати результати тестування і виконувати аналітичні операції.

Перейдемо тепер до створення таблиць. Всі таблиці моєї БД знаходяться в підпапці "Tables" папки " TestsDB " у вікні оглядача об'єктів.

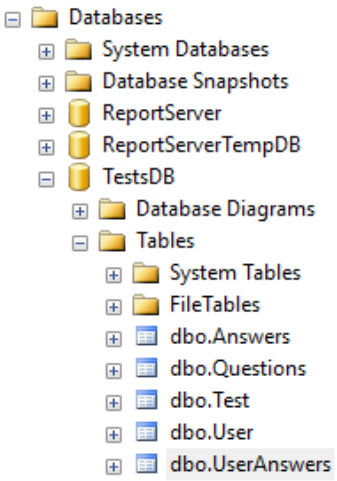

Рис 3.1 – Папка з таблицями бази данних

Створив таблицю "Users". Дана таблиця зберігає інформацію про користувачів, які проходять тестування. Вона використовується для ідентифікації користувача під час виконання тестів, а також для аналізу його відповідей.

Кожен користувач отримує унікальний ідентифікатор (UserID), що дозволяє прив’язувати його відповіді до конкретного тестування.

Ім'я (Username) записується для подальшої обробки результатів тесту та відображення даних адміністраторам.

Завдяки цій таблиці можна зберігати історію тестувань кожного користувача, що дозволяє аналізувати його успішність та прогрес у навчанні.

DESKTOP-3ND188H.TestsDB - dbo.User

UserID	Username
1	Yura
3002	Юрій Гробов
NULL	NULL

Рис 3.2 – Таблиця "Users"

Створив таблицю "Test". Дана таблиця зберігає перелік тестів, доступних у системі, та дозволяє об'єднувати запитання за відповідними категоріями, що значно спрощує управління тестами.

DESKTOP-3ND188H.TestsDB - dbo.Test

TestID	Name
1	Основи програмування
2	Об'єктно-орієнтоване програмування
3	Бази данних
8	Алгоритми
9	Основи Pyhton
1009	Основи HTML
1010	Основи CSS
1011	Основи GIT
NULL	NULL

Рис 3.3 – Таблиця "Test"

Створив таблицю "Questions". Дана таблиця містить усі запитання, які входять до тестів. Вона тісно пов’язана з таблицею Test, що дозволяє групувати питання за темами.

Кожне питання має унікальний ідентифікатор (QuestionID) і належить до певного тесту (TestID). Також у цій таблиці міститься текст питання

(QuestionText) і поле (CorrectAnswer), яке вказує правильний варіант відповіді.

Ця таблиця відіграє важливу роль у системі тестування, оскільки на її основі формується зміст тесту, який проходять користувачі.

DESKTOP-3ND188H....B - dbo.Questions

QuestionID	TestID	QuestionText	CorrectAnswer
1	1	Що таке змінна?	1
2	1	Який оператор використовують для залишку від ділення?	1
3	1	Який тип даних використовується для зберігання тексту?	2
4	2	Що таке клас у програмуванні?	1
5	2	Що таке об'єкт у програмуванні?	2
6	2	Що означає наслідування в ООП?	2
7	3	Що таке первинний ключ у базі даних?	1
8	3	Який SQL-запит використовується для вибірки даних?	1
9	3	Що означає термін "реляційна база даних"?	1
10	8	Що таке алгоритм?	1
11	8	Що означає "складність алгоритму"?	3
12	8	Що таке рекурсія?	1
13	9	Що таке Python?	1
14	9	Як оголосити змінну в Python?	2
15	9	Що робить функція print()?	1
16	1009	Що таке HTML?	1
17	1009	Який тег використовується для заголовків?	1
18	1009	Що таке атрибут у HTML?	1
19	1010	Що таке CSS?	1
20	1010	Як задати колір тексту в CSS?	1
21	1010	Що таке Flexbox?	1
22	1011	Що таке Git?	1
23	1011	Яка команда створює новий репозиторій?	1
24	1011	Яка команда зберігає зміни в репозиторії?	1
NULL	NULL	NULL	NULL

Рис 3.4 – Таблиця "Questions"

Створив таблицю "Answer" . Дана таблиця Answers містить усі варіанти відповідей, які можуть бути використані в тестах. Вона пов'язана з таблицею Questions через поле QuestionID, що дозволяє прив'язувати відповіді до конкретного питання.

Важливою особливістю цієї таблиці є поле Correct, яке вказує, чи є відповідь правильною (1) або неправильною (0). Це дозволяє системі автоматично перевіряти відповіді користувачів.

DESKTOP-3ND188H....sDB - dbo.Answers

id	Answer	Correct	QuestionID
1	Область пам'я...	1	1
2	Місце для фун...	0	1
3	Область пам'я...	1	1
4	%	1	2
5	int	0	3
6	Шаблон для ст...	1	4
7	Змінна	0	5
8	Копіювання кл...	0	6
9	Унікальний іде...	1	7
10	SELECT	1	8
11	База даних з ві...	1	9
12	Послідовність ...	1	10
13	Швидкість роб...	0	11
14	Функція, що в...	1	12
15	Мова програм...	1	13
16	let x = 5	0	14
17	Виводить текст...	1	15
18	Мова для ство...	1	16
19	<h1>	1	17
20	Властивість ел...	1	18
21	Мова стилізації	1	19
22	color:	1	20
23	Метод вирівн...	1	21
24	Система контр...	1	22
25	git init	1	23
26	git commit	1	24
27	Місце для фун...	0	1
28	/	0	2
29	string	1	3
30	Функція	0	4
31	Екземпляр кла...	1	5
32	Передача влас...	1	6
33	Тип даних	0	7
34	UPDATE	0	8
35	Файлова струк...	0	9
36	Функція	0	10
37	Кількість опер...	0	11
38	Цикл	0	12

Рис 3.5 – Таблиця "Answer"

Створив таблицю "UserAnswers". Дана таблиця зберігає інформацію про відповіді, які користувачі надали під час тестування. Вона містить поля, які пов'язують відповіді з користувачем (UserID), конкретним питанням (AnswerID) і часом, коли була дана відповідь (AnswerTime).

DESKTOP-3ND188H....- dbo.UserAnswers

UserAnswerID	AnswerTime	UsersID	AnswerID
1007	2025-01-30 15:0...	5	57
1008	2025-01-30 15:0...	5	58
1009	2025-01-30 15:0...	5	35
1010	2025-01-30 15:2...	6	9
1011	2025-01-30 15:2...	6	10
1012	2025-01-30 15:2...	6	35
1013	2025-01-31 19:1...	7	33
1014	2025-01-31 19:1...	7	10
1015	2025-01-31 19:1...	7	35
1016	2025-01-31 19:4...	8	33
1017	2025-01-31 19:4...	8	34
1018	2025-01-31 19:4...	8	11
1019	2025-01-31 19:4...	9	33
1020	2025-01-31 19:4...	9	10
1021	2025-01-31 19:4...	9	59
1022	2025-01-31 19:4...	10	9
1023	2025-01-31 19:4...	10	10
1024	2025-01-31 19:4...	10	59
1025	2025-01-31 19:4...	11	9

Рис 3.6 – Таблиця "UserAnswers"

3.2 Реалізація програмного коду

Розробка тестувальної системи почалася зі створення класу TestApp, який відповідає за взаємодію користувача з програмою, графічний інтерфейс та обробку даних. Основна мета класу – надати користувачеві можливість вибирати тест, відповідати на запитання та зберігати результати у базі даних.

Для реалізації графічного інтерфейсу використовується бібліотека Tkinter, яка забезпечує зручний спосіб створення віконного додатка. Для роботи з базою даних використовується спеціальний клас Database, що відповідає за отримання тестів, питань і запис результатів.

Метод __init__ ініціалізує головне вікно програми, задає його параметри та підключається до бази даних.

Також отримується список доступних тестів, після чого викликається метод створення елементів інтерфейсу.

```
7   class TestApp:  1 usage
8       def __init__(self, root):
9           self.root = root
10          self.root.title("Тестування")
11          self.root.geometry("1000x1200")
12          self.root.configure(bg="#e3e3e3")
13
14          self.database = Database()
15          self.all_test = self.database.get_all_test()
16          self.all_testname = [test[1] for test in self.all_tes
17
18          self.widgets = []
19
20          self.create_widgets()
```

Рис 3.2.1 – Ініціалізація головного класу TestApp

Наступним кроком створено метод create_widgets(), який формує графічний інтерфейс програми. Він додає заголовок, випадаючий список для вибору тесту та прив'язує подію, яка викликає метод load_test() при виборі тесту користувачем.

```
22      def create_widgets(self):  1 usage
23          title_label = tk.Label(self.root, text="Оберіть тест:", font=("Arial", 16, "bold"), bg="#e3e3e3", fg="#444")
24          title_label.pack(pady=10)
25
26          self.comboExample = ttk.Combobox(self.root, values=self.all_testname, font=("Arial", 14))
27          self.comboExample.pack(pady=10, padx=20, ipadx=10, fill=tk.X)
28          self.comboExample.bind("<<ComboboxSelected>>", self.load_test)
```

Рис 3.2.2 – Метод створення графічного інтерфейсу

Після вибору тесту система має завантажити відповідні питання. Для цього реалізовано метод load_test() , який перед своїм виконанням очищує попередні елементи інтерфейсу за допомогою методу clear_widgets(). Останній проходиться по списку віджетів, видаляє їх і очищає список.

```
30      def clear_widgets(self):  1 usage
31          for widget in self.widgets:
32              widget.destroy()
33          self.widgets.clear()
34
```

Рис 3.2.3 – Метод очищення інтерфейсу

Метод load_test() отримує тест, вибраний користувачем, завантажує всі його питання та варіанти відповідей із бази даних. Кожне питання додається до списку self.questions, щоб у майбутньому мати доступ до вибраних відповідей.

```
35      def load_test(self, event):  1 usage
36          self.clear_widgets()
37
38          test = self.database.get_all_task(self.comboExample.get())
39          self.test_title = test[1]
40          questions = self.database.get_all_questions(test[0])
41          self.questions = []
42
43          for question in questions:
44              data = {
45                  "question": question,
46                  "answers": self.database.get_all_answer(question[0]),
47                  "correct": 0
48              }
49              self.questions.append(data)
```

Рис. 3.2.4 – Метод завантаження тесту

Для візуального відображення питання та варіантів відповідей створено frame, у якому розміщуються заголовок тесту, блоки з питаннями та варіанти відповідей .

```
51          frame = tk.Frame(self.root, bg="#ffffff", padx=30, pady=30, relief=tk.GROOVE, bd=3)
52          frame.pack(pady=20, padx=20, fill=tk.BOTH, expand=True)
53          self.widgets.append(frame)
54
55          title_label = tk.Label(frame, text=self.test_title, font=("Arial", 18, "bold"), fg="#222", bg="#ffffff")
56          title_label.pack(pady=10)
57          self.widgets.append(title_label)
58
59          name_label = tk.Label(frame, text="Ваше ім'я та прізвище:", font=("Arial", 14), bg="#ffffff")
60          name_label.pack(pady=5)
61          self.widgets.append(name_label)
62
63          self.name_entry = tk.Entry(frame, font=("Arial", 14), width=35, bd=3, relief=tk.SUNKEN)
64          self.name_entry.pack(pady=5, ipadx=5, ipady=5)
65          self.widgets.append(self.name_entry)
66
67          self.question_vars = []
68          for i, question in enumerate(self.questions):
69              question_label = tk.Label(frame, text=question["question"][2], font=("Arial", 14, "bold"), bg="#ffffff")
70              question_label.pack(pady=10, anchor="w")
71              self.widgets.append(question_label)
72
73              answer_var = StringVar(value="")
74              self.question_vars.append(answer_var)
75
76              for idx, answer in enumerate(question["answers"]):
77                  radio_button = tk.Radiobutton(frame, text=answer[1], variable=answer_var, value=str(answer[0]),
78                                                font=("Arial", 13), bg="#ffffff")
79                  radio_button.pack(anchor="w", padx=20)
80                  self.widgets.append(radio_button)
81
82          self.submit_button = tk.Button(frame, text="Зберегти результат", font=("Arial", 14, "bold"), bg="#007BFF",
83                                         fg="white", padx=15, pady=8, bd=2, relief=tk.RAISED, cursor="hand2",
84                                         command=self.save_result)
85          self.submit_button.pack(pady=30)
86          self.widgets.append(self.submit_button)
```

Рис. 3.2.5 – Відображення тесту

Завершальним етапом реалізації тестування є збереження результатів користувача. Для цього створено метод save_result(), який спочатку перевіряє, чи було введене ім'я, перед тим як записати відповіді у базу даних.

```
88      def save_result(self):  1 usage
89          name = self.name_entry.get()
90          if not name:
91              messagebox.showerror( title: "Помилка",  message: "Будь ласка, введіть ваше ім'я та прізвище.")
92              return
```

Рис. 3.2.6 – Перевірка введення імені користувача

Після перевірки правильності введення отримується userid, який зберігається у базі даних. Далі збираються всі вибрані відповіді у список та записуються у таблицю результатів .

```
94          userid = self.database.save_user(name)
95          data = []
96          time = datetime.datetime.now()
97
98          for var in self.question_vars:
99              data.append((userid, int(var.get()), time))
100
101         self.database.save_useranswer(data)
102         messagebox.showinfo( title: "Результат",  message: "Ваші відповіді успішно збережені!")
103
```

Рис. 3.2.7 – Метод збереження результатів тесту

3.3 Адміністративна панель

Наступним етапом розробки стала реалізація адміністративної панелі, яка дозволяє переглядати результати тестувань користувачів, здійснювати пошук за іменем та аналізувати невдалі спроби.

Для створення графічного інтерфейсу використовується бібліотека Tkinter, яка забезпечує можливість зручного управління тестовими даними. Для взаємодії з базою даних використовується клас Database, що містить методи отримання інформації про користувачів і тестові результати.

Головне вікно адміністративної панелі створюється у наступному коді. Тут налаштовується розмір, колір фону та заголовок вікна.

```python
import tkinter as tk
from tkinter import ttk
from tkinter.messagebox import showinfo
from database import Database

root = tk.Tk()
root.title('Адмін-панель')
root.geometry('900x550')
root.configure(bg="#2C3E50")
```

Рис. 3.3.1 – Ініціалізація головного вікна адміністративної панелі

Для структурування інтерфейсу використовується контейнер main_frame, який містить усі віджети панелі .

```python
main_frame = tk.Frame(root, bg="#2C3E50")
main_frame.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)
```

Рис. 3.3.2 – Головний контейнер інтерфейсу

Щоб забезпечити можливість пошуку користувачів за іменем, було створено окремий блок search_frame, що містить текстове поле для введення імені та кнопку "Пошук". При натисканні кнопки викликається метод search_user(), який отримує результати тестування користувача з бази даних.

```python
search_frame = tk.Frame(main_frame, bg="#34495E", bd=2, relief="solid")
search_frame.pack(fill=tk.X, pady=(0, 10), padx=5)


tk.Label(search_frame, text="Ім'я користувача:", fg="white", bg="#34495E", font=("Arial", 12, "bold")).pack(side=tk.LEFT, padx=10)
user_name_entry = tk.Entry(search_frame, font=("Arial", 12), bg="#ECF0F1", relief="solid", bd=1)
user_name_entry.pack(side=tk.LEFT, fill=tk.X, expand=True, padx=5, pady=5)


search_button = tk.Button(search_frame, text="Пошук", font=("Arial", 12, "bold"), bg="#E74C3C", fg="white",
                          activebackground="#C0392B", activeforeground="white", relief="flat", padx=10, pady=2 , command= search_user)
search_button.pack(side=tk.RIGHT, padx=10, pady=5)
```

Рис. 3.3.3 – Форма пошуку користувача

Метод search_user() отримує ім'я користувача з введеного поля, виконує SQL-запит до бази даних і відображає знайдені результати у вигляді таблиці.

```python
def search_user():  1 usage
    clear()
    database = Database()
    tree.heading('user_id', text="Кількість балів")
    for contact in database.search_user_test(user_name_entry.get()):
        tree.insert( parent: '', tk.END, values=tuple(contact))
```

Рис. 3.3.4 – Метод пошуку користувача

Щоб адміністратор мав можливість аналізувати результати тестів, було додано дві основні функції: перегляд невдалих спроб і перегляд кількості спроб проходження тесту.

Кнопки для виконання цих функцій розташовані в окремому контейнері.

```
row_frame = tk.Frame(buttons_frame, bg="#2C3E50")
row_frame.pack(fill=tk.X, pady=2)
button1 = tk.Button(row_frame, text="Кнопка 1", width=20, font=("Arial", 12, "bold"),
                    bg="#3498DB", fg="white", activebackground="#2980B9", activeforeground="white", relief="flat", pady=5 , command= incorrect_answer)
button1.pack(side=tk.LEFT, padx=5, pady=5, expand=True)

button3 = tk.Button(row_frame, text="Кнопка 3", width=20, font=("Arial", 12, "bold"),
                    bg="#3498DB", fg="white", activebackground="#2980B9", activeforeground="white", relief="flat", pady=5 , command=requests)
button3.pack(side=tk.LEFT, padx=5, pady=5, expand=True)
```

Рис. 3.3.5 – Кнопки для перегляду результатів тестування

Метод incorrect_answer() отримує список користувачів, які не відповіли правильно на жодне питання.

```
def incorrect_answer():  1 usage
    clear()
    database = Database()
    for contact in database.incorrect_answer_questions():
        tree.insert( parent: '', tk.END, values=(contact[0],contact[1] , 'Не пройшов тест'))
```

Рис. 3.3.6 – Виведення користувачів, які не склали тест

Метод requests() підраховує кількість спроб проходження тестів кожного користувача.

```
def requests():  1 usage
    clear()
    database = Database()
    for contact in database.requests_test():
        tree.insert( parent: '', tk.END, values=tuple(contact))
```

Рис. 3.3.7 – Виведення кількості спроб проходження тесту

Щоб результати тестування було зручно переглядати, було реалізовано таблицю (Treeview), яка містить колонки з іменем користувача, назвою тесту та його унікальним ID.

```
columns = ('user_name', 'test_name', 'user_id')
tree_frame = tk.Frame(main_frame, bg="#34495E", bd=2, relief="solid")
tree_frame.pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

tree = ttk.Treeview(tree_frame, columns=columns, show='headings', selectmode="browse")


style = ttk.Style()
style.configure( style: "Treeview", font=("Arial", 12), background="#ECF0F1", fieldbackground="#ECF0F1", rowheight=25)
style.configure( style: "Treeview.Heading", font=("Arial", 12, "bold"), background="#BDC3C7", foreground="black")


tree.heading('user_name', text="Ім'я користувача")
tree.heading('test_name', text="Назва тесту")
tree.heading('user_id', text="ID користувача")
```

Рис. 3.3.8 – Таблиця для відображення результатів тестування

Також реалізовано прив’язку події вибору до таблиці, щоб при натисканні на запис можна було переглянути детальну інформацію.

```
87  def item_selected(event):  1 usage
88      for selected_item in tree.selection():
89          item = tree.item(selected_item)
90          record = item['values']
91          showinfo(title='Інформація', message=','.join(record))
92
93  tree.bind('<<TreeviewSelect>>', item_selected)
```

Рис. 3.3.9 – Функція відображення детальної інформації про вибраний запис

Останнім елементом є скролбар, який дозволяє переглядати великі списки записів.

```
96   scrollbar = ttk.Scrollbar(tree_frame, orient=tk.VERTICAL, command=tree.yview)
97   tree.configure(yscroll=scrollbar.set)
98
99   # Розміщення таблиці та скролбара
100  tree.pack(fill=tk.BOTH, expand=True, side=tk.LEFT)
101  scrollbar.pack(fill=tk.Y, side=tk.RIGHT)
```

Рис. 3.3.10 – Додавання скролбара до таблиці

3.4 Робота з базою даних

Для підключення до бази даних використовується драйвер ODBC Driver 11 for SQL Server. При створенні об'єкта класу Database здійснюється перевірка, чи підключення вже існує. Якщо ні – створюється нове.

```python
import pyodbc
class Database:  35 usages

    conn = None
    cursor = None

    def __init__(self):
        server = "localhost"
        database = "TestsDB"

        connection_string = f"DRIVER={{ODBC Driver 11 for SQL Server}}; SERVER={server}; DATABASE={database}; Trusted_Connection=yes"
        if Database.conn is None:
            Database.conn = pyodbc.connect(connection_string)
            Database.cursor = Database.conn.cursor()
```

Рис. 3.4.1 – Ініціалізація підключення до бази даних

Для отримання списку доступних тестів використовується метод get_all_test(), який виконує SQL-запит до таблиці Test.

```python
    def get_all_tasks(self):
        Database.cursor.execute("SELECT * FROM Answers ")
        select_data = Database.cursor.fetchall()
        return select_data
```

Рис. 3.4.2 – Отримання списку тестів

Метод get_all_taskss() виконує запит до бази даних, щоб отримати всі записи з таблиці Questions.

```python
    def get_all_taskss(self):
        Database.cursor.execute("SELECT * FROM Questions ")
        select_data = Database.cursor.fetchall()
        return select_data
```

Рис. 3.4.3 – Метод отримання всіх питань із бази даних

Метод get_all_tasksss() виконує SQL-запит для отримання всіх записів із таблиці UserAnswers.

```python
    def get_all_tasksss(self):
        Database.cursor.execute("SELECT * FROM UserAnswers ")
        select_data = Database.cursor.fetchall()
        return select_data
```

Рис. 3.4.4 – Метод отримання всіх відповідей користувачів із бази даних

Метод save_task() використовується для збереження нового питання в базу даних. Він приймає аргумент data, який містить інформацію про питання та варіанти відповідей.

```
33    def save_task(self, data):
34        Database.cursor.execute("INSERT INTO QuestionText,Answer1,Answer2,Answer3,CorrectAnswer, VALUES (?, ?, ?, ?, ?)",data)
35        self.conn.commit()
```

ис. 3.4.5 – Метод збереження нового питання в базу даних

Метод get_all_task() виконує SQL-запит для отримання одного запису з таблиці Test, де значення у стовпці Name відповідає переданому taskid.

```
    def get_all_task(self,taskid):  1 usage
38      Database.cursor.execute("SELECT * FROM Test WHERE Name=? " , (taskid))
39      select_data = Database.cursor.fetchone()
40      return select_data
```

Рис. 3.4.6 – Метод отримання інформації про тест із бази даних

Метод get_all_questions() виконує SQL-запит для отримання всіх питань, що належать до певного тесту, визначеного його TestID.

```
42    def get_all_questions(self,taskid):  1 usage
43        Database.cursor.execute("SELECT * FROM Questions WHERE TestID=? ", (taskid))
44        select_data = Database.cursor.fetchall()
45        return select_data
```

Рис. 3.4.7 – Метод отримання всіх питань тесту з бази даних

Метод get_all_answer() виконує SQL-запит для отримання всіх відповідей, що належать до конкретного питання за його QuestionID.

```
47    def get_all_answer(self,questionid):  1 usage
48        Database.cursor.execute("SELECT * FROM Answers WHERE QuestionID=? ", (questionid))
49        select_data = Database.cursor.fetchall()
50        return select_data
```

Рис. 3.4.8 – Метод отримання всіх відповідей на питання з бази даних

Метод get_all_test() виконує SQL-запит для отримання списку всіх тестів із таблиці Test.

```
52        def get_all_test(self):  1 usage
53            Database.cursor.execute("SELECT * FROM Test")
54            select_data = Database.cursor.fetchall()
55            return select_data
```

Рис. 3.4.9 – Метод отримання списку всіх тестів із бази даних

Метод save_useranswer() використовується для збереження відповідей користувачів у таблиці UserAnswers.

```
57    def save_useranswer(self, data):  1 usage
58        Database.cursor.executemany("INSERT INTO [UserAnswers] ( UsersID,AnswerID,AnswerTime) VALUES   (?, ?, ?)",data)
59        self.conn.commit()
```

Рис. 3.4.10 – Метод збереження відповідей користувачів у базу даних

Метод save_user() використовується для додавання нового користувача в базу даних та отримання його UserID.

```
61    def save_user(self, name):  1 usage
62        Database.cursor.execute("INSERT INTO [User] (Username) OUTPUT INSERTED.UserID VALUES (?)",name)
63        inserted_id = Database.cursor.fetchone()[0]
64        Database.conn.commit()
65        return inserted_id
```

Рис. 3.4.11 – Метод додавання нового користувача в базу даних

Метод search_user_test() використовується для пошуку результатів тестування конкретного користувача за його іменем.

```
def search_user_test(self, name):  1 usage
    Database.cursor.execute(
        "SELECT [User].Username, Test.Name, SUM(Answers.Correct) "
        "FROM UserAnswers "
        "INNER JOIN [User] ON UserAnswers.UsersID = [User].UserID "
        "INNER JOIN [Answers] ON UserAnswers.AnswerID = Answers.id "
        "INNER JOIN [Questions] ON Answers.QuestionID = Questions.QuestionID "
        "INNER JOIN [Test] ON Questions.TestID = Test.TestID "
        "WHERE [User].Username = ? "
        "GROUP BY [User].Username, [Test].Name",
        (name,)
    )

    select_data = Database.cursor.fetchall()
    return select_data
```

Рис. 3.4.12 – Метод пошуку результатів тестування користувача

Метод incorrect_answer_questions() використовується для виявлення користувачів, які не відповіли правильно на жодне питання в тесті.

```
72      def incorrect_answer_questions(self):  1 usage
73          Database.cursor.execute(
74              "SELECT DISTINCT [User].Username, Test.Name "
75              "FROM UserAnswers "
76              "INNER JOIN [User] ON UserAnswers.UsersID = [User].UserID "
77              "INNER JOIN Answers ON UserAnswers.AnswerID = Answers.id "
78              "INNER JOIN Questions ON Answers.QuestionID = Questions.QuestionID "
79              "INNER JOIN Test ON Questions.TestID = Test.TestID "
80              "GROUP BY [User].Username, Test.Name "
81              "HAVING SUM(Answers.Correct) = 0;"
82          )
83
84          select_data = Database.cursor.fetchall()
85          return select_data
```

Рис. 3.4.13 – Метод виявлення користувачів, які не відповіли правильно на жодне питання

Метод requests_test() використовується для підрахунку кількості спроб проходження тесту кожним користувачем.

```
77      def requests_test(self):  1 usage
78          Database.cursor.execute(
79              "SELECT [User].Username, Test.Name, COUNT(*) AS Attempts
80              "FROM UserAnswers "
81              "INNER JOIN [User] ON UserAnswers.UsersID = [User].UserID
82              "INNER JOIN Questions ON UserAnswers.AnswerID = Questions
83              "INNER JOIN Test ON Questions.TestID = Test.TestID "
84              "GROUP BY [User].Username, Test.Name "
85              "ORDER BY Attempts DESC;"
86          )
87
88          select_data = Database.cursor.fetchall()
89          return select_data
```

Рис. 3.4.14 – Метод підрахунку кількості спроб проходження тестів

4. КЕРІВНИЦТВО КОРИСТУВАЧА

4.1 Опис роботи з програмою для користувача

Перед використанням програми необхідно виконати запуск тестувального інтерфейсу.

Відкрити командний рядок або термінал. Перейти до директорії, де знаходиться файл main.py та запустити програму командою python main.py.

```
PS C:\Users\grobo\Desktop\коледж\python> python main.py
```

Рис. 4.1.1 – Запуск програми через командний рядок

Після запуску програми на екрані з’явиться головне вікно, у якому буде запропоновано вибрати тест із випадаючого списку. У випадаючому списку користувач може обрати один із доступних тестів.Програма автоматично завантажить питання, що належать до вибраного тесту.

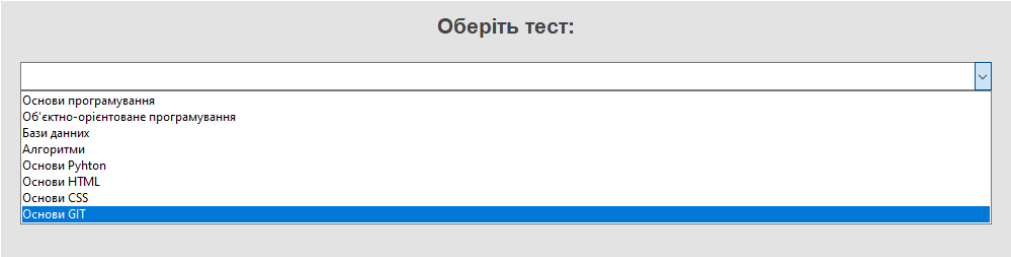

Рис. 4.1.2 – Інтерфейс вибору тесту користувачем

Після вибору тесту користувачеві буде запропоновано ввести своє ім'я та відповісти на серію запитань.У полі "Ім'я користувача" необхідно ввести своє ім'я.Для кожного питання буде запропоновано декілька варіантів відповідей.Користувач має вибрати одну правильну відповідь, натиснувши на радіокнопку біля обраного варіанту.

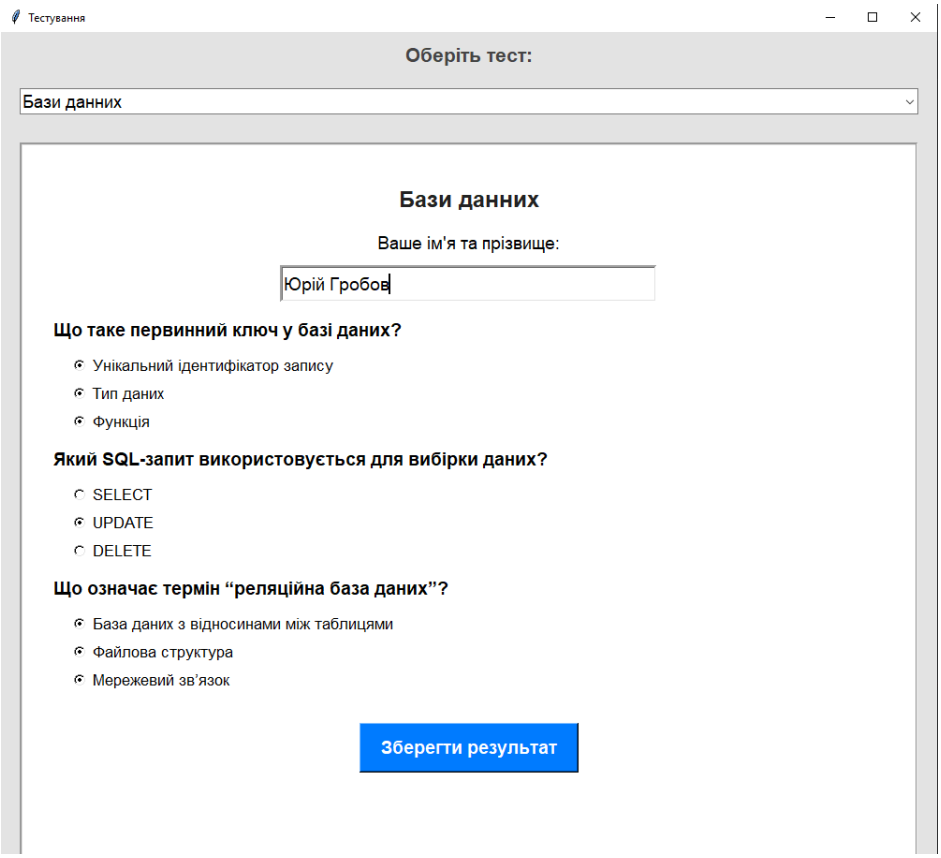

Рис. 4.1.3 – Проходження тесту користувачем

Після завершення тестування програма автоматично збереже відповіді користувача в базі даних.

Програма перевіряє введені дані:

Чи введене ім'я користувача?

Чи обрані відповіді на всі запитання?

Якщо всі дані коректні, відповіді записуються в базу.

Виводиться повідомлення "Ваші відповіді успішно збережені!".

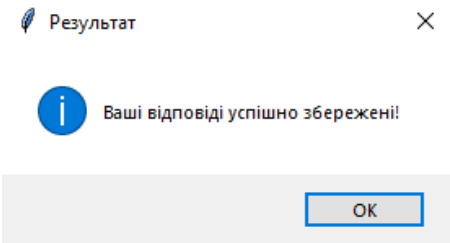

Рис. 4.1.4 – Повідомлення про успішне збереження відповідей

Після завершення роботи з тестами програму можна закрити.Для цього потрібно **закрити вікно програми**.

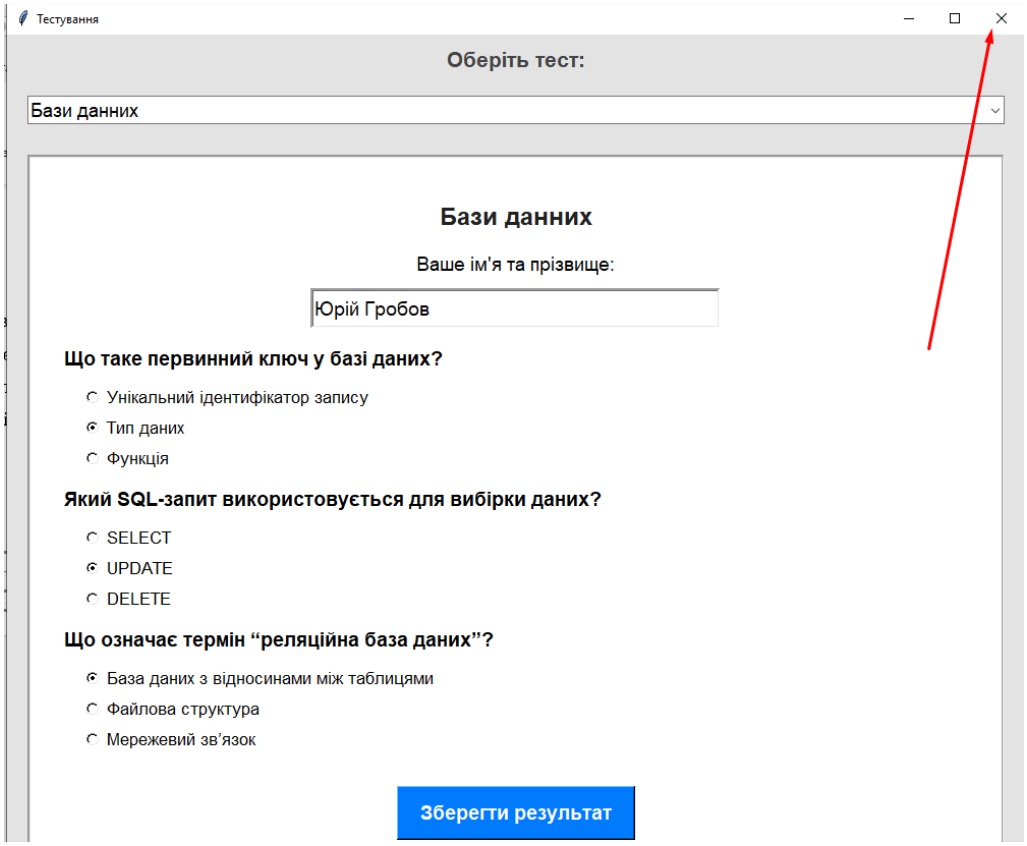

Рис. 4.1.5 – Закриття програми

4.2 Опис роботи з програм для адміністратора

Перед використанням програми необхідно виконати запуск тестувального інтерфейсу.

Відкрити командний рядок або термінал. Перейти до директорії, де знаходиться файл main.py та запустити програму командою python admin.py.

```
PS C:\Users\grobo\Desktop\коледж\python> python admin.py
```

Рис. 4.2.1 – Запуск програми через командний рядок

Після запуску відкриється головне вікно, яке містить такі елементи:

Поле пошуку користувача – дозволяє знайти результати тестування конкретного користувача.

Кнопки управління – надають доступ до перегляду користувачів, які не впоралися із тестами, а також до статистики тестувань.

Таблиця результатів – відображає інформацію про користувачів та їхні тести.

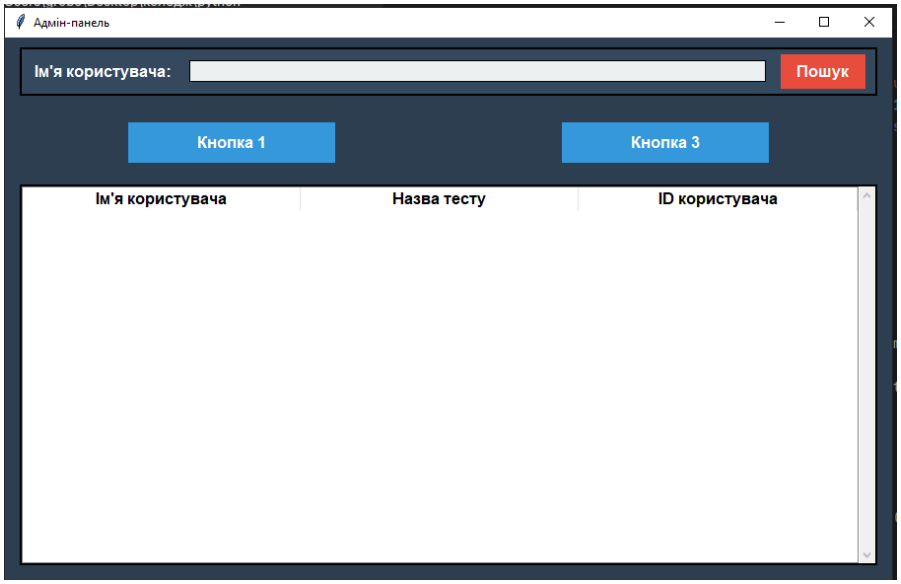

Рис. 4.2.2 – Головне вікно адміністративної панелі

Адміністратор може знайти результати тестування конкретного користувача за його іменем.

У полі "Ім'я користувача" потрібно ввести ім'я користувача.

Натиснути кнопку "Пошук".

В таблиці з'являться результати тестування даного користувача.

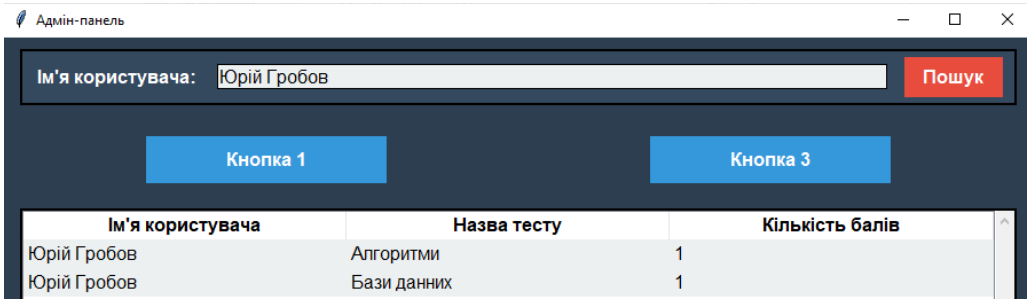

Рис. 4.2.3 – Пошук результатів тестування конкретного користувача

Програма дозволяє знайти користувачів, які не дали жодної правильної відповіді.

Натиснути кнопку "Кнопку1".

У колонці результату буде зазначено "Не пройшов тест".

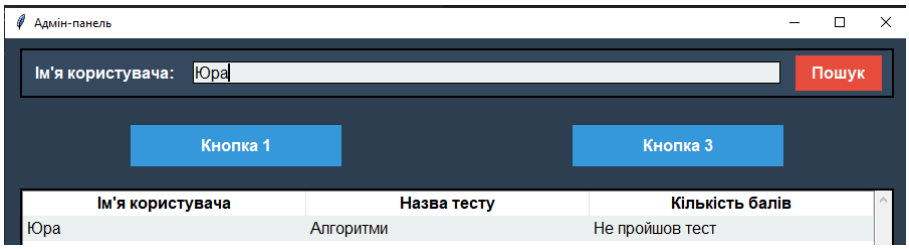

Рис. 4.2.4 – Перегляд користувачів, які не відповіли правильно

Адміністратор може переглянути кількість спроб проходження тестів кожним користувачем.

Натиснути кнопку "Кнопку3".

У таблиці з'являться користувачі та кількість їхніх спроб пройти тест.

В колонці "Кількість спроб" буде відображено число проходжень тесту кожним користувачем.

Рис. 4.2.5 – Перегляд загальної статистики проходження тестів

Після завершення роботи з тестами програму можна закрити.Для цього потрібно **закрити вікно програми**.

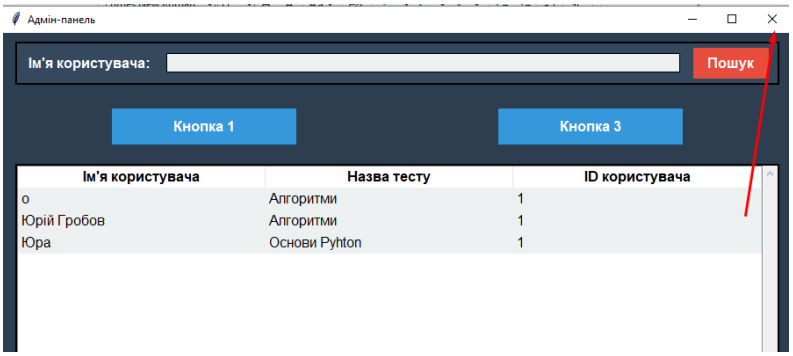

Рис. 4.2.6 – Закриття програми

ВИСНОВКИ

У ході виконання курсового проєкту було проведено аналіз вимог до тестувальної системи, що дозволило визначити основні функціональні можливості майбутнього застосунку. Було розглянуто сучасні підходи до реалізації подібних систем, а також обрано відповідні технології для розробки. В результаті спроєктовано архітектуру програми, визначено структуру бази даних та взаємодію між її компонентами.

Під час розробки програмного забезпечення було створено два основних модулі: клієнтський інтерфейс для проходження тестів та адміністративну панель для аналізу результатів. Графічний інтерфейс реалізовано за допомогою бібліотеки Tkinter, що забезпечує зручну взаємодію користувачів із програмою. Взаємодія із базою даних здійснюється через SQL-запити, які дозволяють отримувати, зберігати та обробляти дані, пов'язані з тестуванням.

На фінальному етапі було проведено тестування всіх функціональних можливостей програми, перевірено коректність виконання SQL-запитів, обробку відповідей користувачів, збереження та оновлення інформації в базі даних. Також було створено керівництво користувача, що пояснює принцип роботи застосунку та порядок його використання.

Результатом виконаної роботи стала повноцінна тестувальна система, яка забезпечує автоматизацію процесу перевірки знань, зручне управління тестами та аналіз результатів. У процесі розробки здобуто практичні навички проєктування програмного забезпечення, роботи з базами даних, використання мови Python для створення застосунків та розробки графічного інтерфейсу.

СПИСОК ЛІТЕРАТУРИ

1. Head-First Python, 2nd edition: Paul Barry. - Sebastopol, California, U.S.:O'Reilly Media, 2016. – 622 с.

2. Think Python: How to Think Like a Computer Scientist, 2nd edition: Allen B.Downey. - Sebastopol, California, U.S.: O'Reilly Media, 2015. – 292 с.

3. Python.org: веб-сайт. URL: https://www.python.org

4. Python Tutorial: веб-сайт. URL: https://www.w3schools.com/python/

5. Learn to become a modern Python developer: веб-сайт. URL: https://roadmap.sh/python/ (дата звернення: 02.01.2023)

6. W3schoolsUA: веб-сайт. URL: https://w3schoolsua.github.io/

7. Створення й редагування файлів CSV: веб-сайт. URL: https://support.microsoft.com/uk-ua/office/

8. About Python: веб-сайт. URL: https://acode.com.ua/intro-python/

ДОДАТКИ

Додаток А

```
import tkinter as tk
from tkinter import messagebox, StringVar, ttk
import datetime
from database import Database


class TestApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Тестування")
        self.root.geometry("1000x1200")
        self.root.configure(bg="#e3e3e3")

        self.database = Database()
        self.all_test = self.database.get_all_test()
        self.all_testname = [test[1] for test in self.all_test]

        self.widgets = []

        self.create_widgets()

    def create_widgets(self):
        title_label = tk.Label(self.root, text="Оберіть тест:", font=("Arial", 16,
"bold"), bg="#e3e3e3", fg="#444")
        title_label.pack(pady=10)
        self.comboExample = ttk.Combobox(self.root, values=self.all_testname,
font=("Arial", 14))
```

```
        self.comboExample.pack(pady=10, padx=20, ipadx=10, fill=tk.X)
        self.comboExample.bind("<<ComboboxSelected>>", self.load_test)

    def clear_widgets(self):
        for widget in self.widgets:
            widget.destroy()
        self.widgets.clear()

    def load_test(self, event):
        self.clear_widgets()

        test = self.database.get_all_task(self.comboExample.get())
        self.test_title = test[1]
        questions = self.database.get_all_questions(test[0])
        self.questions = []

        for question in questions:
            data = {
                "question": question,
                "answers": self.database.get_all_answer(question[0]),
                "correct": 0
            }
            self.questions.append(data)

        frame = tk.Frame(self.root, bg="#ffffff", padx=30, pady=30,
relief=tk.GROOVE, bd=3)
        frame.pack(pady=20, padx=20, fill=tk.BOTH, expand=True)
        self.widgets.append(frame)
```

```
        title_label = tk.Label(frame, text=self.test_title, font=("Arial", 18, "bold"),
fg="#222", bg="#ffffff")
        title_label.pack(pady=10)
        self.widgets.append(title_label)

        name_label = tk.Label(frame, text="Ваше ім'я та прізвище:", font=("Arial",
14), bg="#ffffff")
        name_label.pack(pady=5)
        self.widgets.append(name_label)

        self.name_entry = tk.Entry(frame, font=("Arial", 14), width=35, bd=3,
relief=tk.SUNKEN)
        self.name_entry.pack(pady=5, ipadx=5, ipady=5)
        self.widgets.append(self.name_entry)

        self.question_vars = []
        for i, question in enumerate(self.questions):
            question_label = tk.Label(frame, text=question["question"][2],
font=("Arial", 14, "bold"), bg="#ffffff")
            question_label.pack(pady=10, anchor="w")
            self.widgets.append(question_label)

            answer_var = StringVar(value="")
            self.question_vars.append(answer_var)

            for idx, answer in enumerate(question["answers"]):
                radio_button = tk.Radiobutton(frame, text=answer[1],
variable=answer_var, value=str(answer[0]),
```

```
                                font=("Arial", 13), bg="#ffffff")
            radio_button.pack(anchor="w", padx=20)

            self.widgets.append(radio_button)

        self.submit_button = tk.Button(frame, text="Зберегти результат",
font=("Arial", 14, "bold"), bg="#007BFF",
                               fg="white", padx=15, pady=8, bd=2, relief=tk.RAISED,
cursor="hand2",
                               command=self.save_result)
        self.submit_button.pack(pady=30)
        self.widgets.append(self.submit_button)

    def save_result(self):
        name = self.name_entry.get()
        if not name:
            messagebox.showerror("Помилка", "Будь ласка, введіть ваше ім'я та
прізвище.")
            return
        userid = self.database.save_user(name)
        data = []
        time = datetime.datetime.now()
        for var in self.question_vars:
            data.append((userid, int(var.get()), time))
        self.database.save_useranswer(data)
        messagebox.showinfo("Результат", "Ваші відповіді успішно збережені!")
root = tk.Tk()
app = TestApp(root)
```

```
root.mainloop()
```

Додаток Б

```
import tkinter as tk
from tkinter import ttk
from tkinter.messagebox import showinfo
from database import Database

root = tk.Tk()
root.title('Адмін-панель')
root.geometry('900x550')  # Збільшене вікно
root.configure(bg="#2C3E50")  # Фон головного вікна

# Головний контейнер
main_frame = tk.Frame(root, bg="#2C3E50")
main_frame.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)
def clear ():
    for a in tree.get_children(tree.focus()):
        tree.delete(a)

def search_user():
    clear()
    database = Database()
    tree.heading('user_id', text="Кількість балів")
    for contact in database.search_user_test(user_name_entry.get()):
        tree.insert('', tk.END, values=tuple(contact))

# Контейнер для пошуку
search_frame = tk.Frame(main_frame, bg="#34495E", bd=2, relief="solid")
```

```
search_frame.pack(fill=tk.X, pady=(0, 10), padx=5)
# Поле "Ім'я користувача"
tk.Label(search_frame, text="Ім'я користувача:", fg="white", bg="#34495E",
font=("Arial", 12, "bold")).pack(side=tk.LEFT, padx=10)
user_name_entry = tk.Entry(search_frame, font=("Arial", 12), bg="#ECF0F1",
relief="solid", bd=1)
user_name_entry.pack(side=tk.LEFT, fill=tk.X, expand=True, padx=5, pady=5)

# Кнопка "Пошук"
search_button = tk.Button(search_frame, text="Пошук", font=("Arial", 12,
"bold"), bg="#E74C3C", fg="white",
                          activebackground="#C0392B", activeforeground="white",
relief="flat", padx=10, pady=2 , command= search_user)
search_button.pack(side=tk.RIGHT, padx=10, pady=5)

# Контейнер для кнопок
buttons_frame = tk.Frame(main_frame, bg="#2C3E50")
buttons_frame.pack(fill=tk.X, pady=10)

def incorrect_answer():
    clear()
    database = Database()
    for contact in database.incorrect_answer_questions():
        tree.insert('', tk.END, values=(contact[0],contact[1] , 'Не пройшов тест'))

def requests():
    clear()
    database = Database()
    for contact in database.requests_test():
```

```
        tree.insert('', tk.END, values=tuple(contact))
# Додавання 3 кнопок в один ряд
row_frame = tk.Frame(buttons_frame, bg="#2C3E50")
row_frame.pack(fill=tk.X, pady=2)
button1 = tk.Button(row_frame, text="Кнопка 1", width=20, font=("Arial", 12,
"bold"),
                    bg="#3498DB", fg="white", activebackground="#2980B9",
activeforeground="white", relief="flat", pady=5 , command= incorrect_answer)
button1.pack(side=tk.LEFT, padx=5, pady=5, expand=True)

button3 = tk.Button(row_frame, text="Кнопка 3", width=20, font=("Arial", 12,
"bold"),
                    bg="#3498DB", fg="white", activebackground="#2980B9",
activeforeground="white", relief="flat", pady=5 , command=requests)
button3.pack(side=tk.LEFT, padx=5, pady=5, expand=True)


# Створення таблиці (Treeview)
columns = ('user_name', 'test_name', 'user_id')
tree_frame = tk.Frame(main_frame, bg="#34495E", bd=2, relief="solid")
tree_frame.pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

tree = ttk.Treeview(tree_frame, columns=columns, show='headings',
selectmode="browse")

# Стилізація таблиці
style = ttk.Style()
style.configure("Treeview", font=("Arial", 12), background="#ECF0F1",
fieldbackground="#ECF0F1", rowheight=25)
```

```
style.configure("Treeview.Heading", font=("Arial", 12, "bold"),
background="#BDC3C7", foreground="black")

# Заголовки колонок
tree.heading('user_name', text="Ім'я користувача")
tree.heading('test_name', text="Назва тесту")
tree.heading('user_id', text="ID користувача")



# Прив'язка події вибору
def item_selected(event):
    for selected_item in tree.selection():
        item = tree.item(selected_item)
        record = item['values']
        showinfo(title='Інформація', message=','.join(record))

tree.bind('<<TreeviewSelect>>', item_selected)

# Додаємо скролбар
scrollbar = ttk.Scrollbar(tree_frame, orient=tk.VERTICAL, command=tree.yview)
tree.configure(yscroll=scrollbar.set)

# Розміщення таблиці та скролбара
tree.pack(fill=tk.BOTH, expand=True, side=tk.LEFT)
scrollbar.pack(fill=tk.Y, side=tk.RIGHT)

# Запуск додатку
root.mainloop()
```

Додаток В

```
        from tkinter.constants import INSERT

import pyodbc
class Database:

    conn = None
    cursor = None

    def __init__(self):
        server = "localhost"
        database = "TestsDB"

        connection_string = f"DRIVER={{ODBC Driver 11 for SQL Server}};
SERVER={server}; DATABASE={database}; Trusted_Connection=yes"
        if Database.conn is None:
            Database.conn = pyodbc.connect(connection_string)
            Database.cursor = Database.conn.cursor()

    def get_all_tasks(self):
        Database.cursor.execute("SELECT * FROM Answers ")
        select_data = Database.cursor.fetchall()
        return select_data

    def get_all_taskss(self):
        Database.cursor.execute("SELECT * FROM Questions ")
        select_data = Database.cursor.fetchall()
        return select_data
```

```
    def get_all_tasksss(self):
        Database.cursor.execute("SELECT * FROM UserAnswers ")
        select_data = Database.cursor.fetchall()
        return select_data

    def save_task(self, data):
        Database.cursor.execute("INSERT INTO
QuestionText,Answer1,Answer2,Answer3,CorrectAnswer, VALUES (?, ?, ?, ?,
?)",data)
        self.conn.commit()

    def get_all_task(self,taskid):
        Database.cursor.execute("SELECT * FROM Test WHERE Name=? " ,
(taskid))
        select_data = Database.cursor.fetchone()
        return select_data

    def get_all_questions(self,taskid):
        Database.cursor.execute("SELECT * FROM Questions WHERE TestID=? ",
(taskid))
        select_data = Database.cursor.fetchall()
        return select_data

    def get_all_answer(self,questionid):
        Database.cursor.execute("SELECT * FROM Answers WHERE
QuestionID=? ", (questionid))
        select_data = Database.cursor.fetchall()
        return select_data
```

```
    def get_all_test(self):
        Database.cursor.execute("SELECT * FROM Test")
        select_data = Database.cursor.fetchall()
        return select_data

    def save_useranswer(self, data):
        Database.cursor.executemany("INSERT INTO [UserAnswers] (
UsersID,AnswerID,AnswerTime) VALUES   (?, ?, ?)",data)
        self.conn.commit()

    def save_user(self, name):
        Database.cursor.execute("INSERT INTO [User] (Username) OUTPUT
INSERTED.UserID VALUES (?)",name)
        inserted_id = Database.cursor.fetchone()[0]
        Database.conn.commit()
        return inserted_id

    def search_user_test(self,name):
        Database.cursor.execute("SELECT [User].Username, Test.Name, sum
(Answers.Correct) from UserAnswers  INNER JOIN [User] ON
UserAnswers.UsersID=[User].UserID INNER JOIN [Answers] ON
UserAnswers.AnswerID=[Answers].id INNER JOIN [Questions] ON
Answers.QuestionID=[Questions].QuestionID INNER JOIN [Test] ON
Questions.TestID=[Test].TestID WHERE [User].Username= ? GROUP BY
[User].Username,  [Test].Name ", (name))
        select_data = Database.cursor.fetchall()
        return select_data
```

```
    def incorrect_answer_questions(self):
        Database.cursor.execute(" SELECT DISTINCT [User].Username, Test.Name
FROM UserAnswers INNER JOIN [User] ON UserAnswers.UsersID =
[User].UserID INNER JOIN Answers ON UserAnswers.AnswerID = Answers.id
INNER JOIN Questions ON Answers.QuestionID = Questions.QuestionID INNER
JOIN Test ON Questions.TestID = Test.TestID GROUP BY [User].Username,
Test.Name HAVING SUM(Answers.Correct) = 0;")
        select_data = Database.cursor.fetchall()
        return select_data

    def requests_test(self):
        Database.cursor.execute("SELECT [User].Username, Test.Name, COUNT(*)
AS Attempts FROM UserAnswers INNER JOIN [User] ON UserAnswers.UsersID
= [User].UserID INNER JOIN Questions ON UserAnswers.AnswerID =
Questions.QuestionID INNER JOIN Test ON Questions.TestID = Test.TestID
GROUP BY [User].Username, Test.Name ORDER BY Attempts DESC; " )
        select_data = Database.cursor.fetchall()
        return select_data
```